ROYAUME DU MAROC
Université Abdelmalek Essaâdi
École Nationale des Sciences Appliquées d'Al Hoceima

Rapport : TP Numéro 3

Initiation à la Programmation Orientée Objet (Java)

Java / POO

Réalisé par :
Yazid TAHIRI ALAOUI
Filière : 1ère année – Ingénierie Données (ID1)
Encadré par : Pr. Mohamed CHERRADI

Année universitaire : 2025 – 2026

Résumé

Ce rapport présente les solutions de la troisième série de Travaux Pratiques (TP3) en langage Java. Au cours de ce TP, nous avons mis en œuvre les concepts fondamentaux de la programmation orientée objet tels que l'encapsulation, les méthodes d'instance, et l'interaction entre les objets.

Les exercices couvrent une variété de classes allant de la modélisation d'entités simples (Point, Rectangle, Compte Bancaire) jusqu'à des systèmes avec une logique métier plus poussée comme la gestion de stocks (Produit) ou des mini-jeux (Jeu Devinette).

Mots-clés : Java, POO, Classes, Objets, Méthodes, Encapsulation.

Table des matières

Résumé . . . . . **i**

1 Travaux Pratiques N° 3 . . . . . **1**

1.1 Exercice 1 : Classe `Point` 1

1.2 Exercice 2 : Classe `Voiture` 2

1.3 Exercice 3 : Classe `CompteBancaire` 3

1.4 Exercice 4 : Classe `Rectangle` 5

1.5 Exercice 5 : Classe `Etudiant` 6

1.6 Exercice 6 : Classe `JeuDevinette` 7

1.7 Exercice 7 : Classe `NombreComplexe` 9

1.8 Exercice 8 : Classe `Produit` 10

Conclusion **13**

Chapitre 1

Travaux Pratiques N° 3

Ce chapitre regroupe l'ensemble des exercices réalisés lors du TP3. Pour chaque exercice, l'énoncé et l'objectif de la classe sont décrits, suivis de l'implémentation en Java.

1.1 | Exercice 1 : Classe `Point`

Le premier exercice consiste à créer une classe `Point` modélisant un point dans un plan booléen 2D avec les coordonnées `x` et `y`. La classe doit inclure des méthodes pour :

- Afficher les coordonnées du point.
- Déplacer le point selon des valeurs de décalage données.
- Calculer la distance entre deux points donnés.

```java
import java.security.PublicKey;
import java.util.Scanner;

public class Point {
    double x;
    double y;
    public Point(){}
    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public void afficher(Point p){
        System.out.println("x: " + p.x + " y: " + p.y);

    }
    public void deplacer(double a, double b){
        x += a;
        y += b;

    }
    public double distance(Point p){
        double dist = Math.sqrt(p.x*p.x + p.y*p.y);
        return dist;
    }
```

```java

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter X coordinate: ");
        double x = sc.nextInt();
        System.out.print("Enter Y coordinate: ");
        double y = sc.nextInt();
        Point p = new Point(x, y);
        p.afficher(p);
        System.out.print("Enter le deplacement de x : ");
        double a = sc.nextDouble();
        System.out.print("Enter le deplacement de Y : ");
        double b = sc.nextDouble();
        p.deplacer(a, b);
        p.afficher(p);
        double distance = p.distance(p);
        System.out.println("Distance: " + distance);
        sc.close();




    }
}
```

Listing 1.1 – Implémentation de la classe Point

1.2 | Exercice 2 : Classe `Voiture`

Il s'agit de définir une classe `Voiture` caractérisée par sa marque et sa vitesse. Les méthodes à implémenter sont :

- `accelerer()` pour augmenter la vitesse selon un incrément.
- `freiner()` pour décrémenter la vitesse, tout en s'assurant qu'elle ne devient pas négative.

```java
import java.util.Scanner;

public class Voiture {
    String marque;
    double vitesse;
    public Voiture(String marque, double vitesse) {
        this.marque = marque;
        this.vitesse = vitesse;
    }

    public void accelerer(Voiture v, double increment) {
        v.vitesse += increment;
        System.out.println("Acceleration:La vitesse de la " + v.
           marque + " est maintenant de " + v.vitesse + " km/h.");
    }
```

```java

    public void freiner(Voiture v, double decrement) {
        if (v.vitesse >= decrement) {
            v.vitesse -= decrement;
        } else {
            v.vitesse = 0;
        }
        System.out.println("Freinage:La vitesse de la " + v.
           marque + " est maintenant de " + v.vitesse + " km/h.");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("--- CREATION DE LA VOITURE ---");
        System.out.print("Enter la marque de la voiture : ");
        String marque = sc.next();
        System.out.print("Enter la vitesse initiale : ");
        double vitesse = sc.nextDouble();

        Voiture v = new Voiture(marque, vitesse);

        System.out.print("\nEnter la valeur d'acceleration : ");
        double valeurAcc = sc.nextDouble();
        v.accelerer(v, valeurAcc);

        System.out.print("\nEnter la valeur de freinage : ");
        double valeurFrein = sc.nextDouble();
        v.freiner(v, valeurFrein);

        sc.close();
    }
}
```

Listing 1.2 – Implémentation de la classe Voiture

1.3 | Exercice 3 : Classe `CompteBancaire`

Cet exercice a pour but de modéliser un compte bancaire avec un `numero` et un `solde`. Il faut coder les opérations suivantes :

- Déposer de l'argent.
- Retirer de l'argent (en vérifiant au préalable que le solde est suffisant).
- Transférer un montant vers un autre compte destinataire.
- Afficher le solde du compte courant.

```java
import java.util.Scanner;

public class CompteBancaire {
    long numero;
```

```java
    double solde;

    public CompteBancaire(long numero, double solde) {
        this.numero = numero;
        this.solde = solde;
    }

    public void deposer(CompteBancaire c, double n_deposer) {
        c.solde += n_deposer;
        System.out.println("Depot effectue avec succes.");
    }

    public void retirer(CompteBancaire c, double n_retirer) {
        if (c.solde >= n_retirer) {
            c.solde -= n_retirer;
            System.out.println("Retrait effectue avec succes.");
        } else {
            System.out.println("Erreur : Solde insuffisant pour
               ce retrait.");
        }
    }

    public void transfer(CompteBancaire c, CompteBancaire d,
       double n_transf) {
        if (c.solde >= n_transf) {
            c.solde -= n_transf;
            d.solde += n_transf;
            System.out.println("Transfert effectue avec succes
               vers le compte " + d.numero);
        } else {
            System.out.println("Erreur : Solde insuffisant pour
               ce transfert.");
        }
    }

    public void affichersolde(CompteBancaire c) {
        System.out.println("-> Solde actuel du compte " + c.
           numero + " : " + c.solde);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("--- CREATION COMPTE 1 ---");
        System.out.print("Entrer le numero du compte : ");
        long numero = sc.nextLong();
        System.out.print("Entrer le solde initial : ");
        double solde = sc.nextDouble();
        CompteBancaire c = new CompteBancaire(numero, solde);
        c.affichersolde(c);

```

```java
        System.out.print("\nEntrer la somme a deposer : ");
        double n_deposer = sc.nextDouble();
        c.deposer(c, n_deposer);
        c.affichersolde(c);

        System.out.print("\nEntrer la somme a retirer : ");
        double n_retirer = sc.nextDouble();
        c.retirer(c, n_retirer);
        c.affichersolde(c);

        System.out.println("\n--- CREATION COMPTE 2 (Destinataire
           ) ---");
        System.out.print("Entrer le numero du compte cible : ");
        long numero2 = sc.nextLong();
        System.out.print("Entrer le solde initial du compte cible
            : ");
        double solde2 = sc.nextDouble();
        CompteBancaire d = new CompteBancaire(numero2, solde2);

        System.out.print("\nEntrer la somme a transferer du
           Compte 1 vers le Compte 2 : ");
        double n_transf = sc.nextDouble();
        c.transfer(c, d, n_transf);

        System.out.println("\n--- BILAN FINAL ---");
        c.affichersolde(c);
        d.affichersolde(d);

        sc.close();
    }
}
```

Listing 1.3 – Implémentation de la classe CompteBancaire

1.4 | Exercice 4 : Classe `Rectangle`

Dans cet exercice, il faut créer une classe `Rectangle` possédant une `largeur` et une `hauteur`. Des méthodes doivent permettre de :

- Calculer et retourner la surface.
- Calculer et retourner le périmètre.

```java
import java.util.Scanner;

public class Rectangle {

    double largeur;
    double hauteur;
    public Rectangle(double largeur, double hauteur){
        this.largeur = largeur;
        this.hauteur = hauteur;
```

```java
    }

    public double surface(Rectangle r){
        double surface = r.largeur * r.hauteur;
        return surface;
    }
    public double perimetre(Rectangle r){
        double perimetre = r.hauteur * 2 + r.largeur *2;
        return perimetre;
    }
    public static void main (String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter largeur du rectangle : ");
        double x = sc.nextDouble();
        System.out.print("Enter hauteur du rectangle : ");
        double y = sc.nextDouble();
        Rectangle r = new Rectangle(x,y);
        double surf =r.surface(r);
        System.out.println("surface : " + surf);
        double peri = r.perimetre(r);
        System.out.println("perimetre : " + peri);
        sc.close();
    }
}
```

Listing 1.4 – Implémentation de la classe Rectangle

1.5 | Exercice 5 : Classe Etudiant

L'objectif est de structurer les informations d'un `Etudiant` (CNE et note) et d'y ajouter le comportement suivant :

- Renvoyer une chaîne sous la forme `"CNE : ... | Note : ..."` à travers une méthode personnalisée `toString()`.
- Vérifier si l'étudiant est admis (`note` $\geq$ `10`) grâce à la méthode `estAdmis()`.

```java
import java.util.Scanner;

public class Etudiant {

    String cne;
    double note;

    public Etudiant(String cne, double note) {
        this.cne = cne;
        this.note = note;
    }

    public String toString() {
        return "CNE : " + this.cne + " | Note : " + this.note;
    }
```

```java

    public boolean estAdmis(Etudiant e) {
        boolean admis = e.note >= 10.0;
        return admis;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter CNE de l'etudiant : ");
        String cneInput = sc.next();

        System.out.print("Enter note de l'etudiant : ");
        double noteInput = sc.nextDouble();

        Etudiant e = new Etudiant(cneInput, noteInput);

        System.out.println(e.toString());

        boolean admis = e.estAdmis(e);
        if (admis) {
            System.out.println("L'etudiant est admis.");
        } else {
            System.out.println("L'etudiant n'est pas admis.");
        }

        sc.close();
    }
}
```

Listing 1.5 – Implémentation de la classe Etudiant

1.6 | Exercice 6 : Classe `JeuDevinette`

Ce programme simule un jeu où l'utilisateur doit deviner un `nombreSecret`. La classe permet de :

- Vérifier la proposition de l'utilisateur par rapport au nombre généré aléatoirement en donnant des indices ("plus grand", "plus petit", "trouvé").
- Afficher un score final dépendant du nombre de `tentatives`.

```java
import java.util.Scanner;

public class JeuDevinette {
    int nombreSecret;
    int tentatives;

    public JeuDevinette(int nombreSecret) {
        this.nombreSecret = nombreSecret;
        this.tentatives = 0;
    }
```

```java

    public boolean verifierNombre(JeuDevinette jeu, int essai) {
        jeu.tentatives++;

        if (essai < jeu.nombreSecret) {
            System.out.println("kber !");
            return false;
        } else if (essai > jeu.nombreSecret) {
            System.out.println("sgher !");
            return false;
        } else {
            System.out.println("tbarkellah 3lik !");
            return true;
        }
    }

    public void afficherScore(JeuDevinette jeu) {
        int score = 100 - (jeu.tentatives * 10);
        if (score < 0) {
            score = 0;
        }
        System.out.println("Nombre de tentatives : " + jeu.
           tentatives);
        System.out.println("Votre score final : " + score + "/100
           ");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int secret = (int)(Math.random() * 10) + 1;
        JeuDevinette jeu = new JeuDevinette(secret);

        System.out.println("--- JEU DE DEVINETTE ---");
        System.out.println("J'ai choisi un nombre entre 1 et 10.
           A vous de jouer !");

        boolean trouve = false;

        while (!trouve) {
            System.out.print("\nEntrez un nombre : ");
            int essai = sc.nextInt();

            trouve = jeu.verifierNombre(jeu, essai);
        }

        System.out.println("-------------------------");
        jeu.afficherScore(jeu);

        sc.close();
    }
```

```
}
```

Listing 1.6 – Implémentation de la classe JeuDevinette

1.7 | Exercice 7 : Classe NombreComplexe

Il s'agit de définir une classe `NombreComplexe` possédant une composante `reel` et une composante `imaginaire`. L'objectif est d'implémenter les opérations mathématiques suivantes :

- L'addition de deux nombres complexes (séparément sur la partie réelle et imaginaire).
- La multiplication complexe.
- L'affichage du nombre au format usuel $a + ib$ ou $a - ib$.

```java
import java.util.Scanner;

public class NombreComplexe {
    double reel;
    double imaginaire;

    public NombreComplexe(double reel, double imaginaire) {
        this.reel = reel;
        this.imaginaire = imaginaire;
    }
    public NombreComplexe addition(NombreComplexe z1,
       NombreComplexe z2) {
        double sommeReel = z1.reel + z2.reel;
        double sommeImag = z1.imaginaire + z2.imaginaire;
        return new NombreComplexe(sommeReel, sommeImag);
    }

    public NombreComplexe multiplication(NombreComplexe z1,
       NombreComplexe z2) {
        double multReel = (z1.reel * z2.reel) - (z1.imaginaire *
           z2.imaginaire);
        double multImag = (z1.reel * z2.imaginaire) + (z1.
           imaginaire * z2.reel);
        return new NombreComplexe(multReel, multImag);
    }

    public void afficher(NombreComplexe z) {
        if (z.imaginaire >= 0) {
            System.out.println(z.reel + " + " + z.imaginaire + "i
               ");
        } else {
            System.out.println(z.reel + " - " + Math.abs(z.
               imaginaire) + "i");
        }
    }

```

```java
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter partie reelle du premiere nombre : ");
        double r1 = sc.nextDouble();
        System.out.print("Enter partie imaginaire du premiere nombre : ");
        double i1 = sc.nextDouble();
        NombreComplexe z1 = new NombreComplexe(r1, i1);

        System.out.print("Enter partie reelle du deusieme nombre : ");
        double r2 = sc.nextDouble();
        System.out.print("Enter partie imaginaire du deusieme nombre : ");
        double i2 = sc.nextDouble();
        NombreComplexe z2 = new NombreComplexe(r2, i2);

        NombreComplexe resultatAdd = z1.addition(z1, z2);
        NombreComplexe resultatMult = z1.multiplication(z1, z2);

        System.out.print("\nz1 = ");
        z1.afficher(z1);
        System.out.print("z2 = ");
        z2.afficher(z2);

        System.out.print("\nAddition = ");
        resultatAdd.afficher(resultatAdd);

        System.out.print("Multiplication = ");
        resultatMult.afficher(resultatMult);

        sc.close();
    }
}
```

Listing 1.7 – Implémentation de la classe NombreComplexe

1.8 | Exercice 8 : Classe `Produit`

Le dernier exercice implémente un système de gestion d'un stock de produits. Chaque `Produit` comprend : un identifiant, nom, description, prix et quantité. Les fonctionnalités principales sont :

- Ajouter une quantité suite à un arrivage si les identifiants correspondent.
- Modifier les détails d'un produit.
- Réinitialiser ou supprimer un produit de manière définitive.
- Récupérer ou afficher les attributs du produit en question.

```java
import java.util.Scanner;

public class Produit {
    int id;
    String nom;
    String description;
    double prix;
    int quantite;

    public Produit(int id, String nom, String description, double prix, int quantite) {
        this.id = id;
        this.nom = nom;
        this.description = description;
        this.prix = prix;
        this.quantite = quantite;
    }

    public void ajouterProd(Produit p, Produit np) {
        if (p.id == np.id) {
            p.quantite += np.quantite;
            System.out.println("Stock mis a jour. Nouvelle quantite de " + p.nom + " : " + p.quantite);
        } else {
            System.out.println("Erreur : Impossible d'additionner des produits differents.");
        }
    }

    public void modifierProd(Produit p, Produit np) {
        if (p.id == np.id) {
            p.nom = np.nom;
            p.description = np.description;
            p.prix = np.prix;
            p.quantite = np.quantite;
        }
    }

    public void supprimerProd(Produit p) {
        p.id = 0;
        p.nom = "";
        p.description = "";
        p.prix = 0.0;
        p.quantite = 0;
    }

    public void getProduitByID(int id) {
        if (this.id == id) {
            this.afficher(this);
        } else {
            System.out.println("Produit non trouve.");
```

```java
        }
    }

    public void getAllProduits() {
        if (this.id != 0) {
            this.afficher(this);
        }
    }

    public void afficher(Produit p) {
        System.out.println("ID: " + p.id + " | Nom: " + p.nom + "
            | Prix: " + p.prix + " | Quantite: " + p.quantite);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Entrer ID : ");
        int id1 = sc.nextInt();
        System.out.print("Entrer le nom : ");
        String nom1 = sc.next();
        System.out.print("Entrer la description : ");
        String desc1 = sc.next();
        System.out.print("Entrer le prix : ");
        double prix1 = sc.nextDouble();
        System.out.print("Entrer la quantite : ");
        int qte1 = sc.nextInt();

        Produit p1 = new Produit(id1, nom1, desc1, prix1, qte1);
        p1.afficher(p1);
        System.out.print("Entrer la quantite reue : ");
        int qte2 = sc.nextInt();

        Produit arrivage = new Produit(id1, nom1, desc1, prix1,
           qte2);

        p1.ajouterProd(p1, arrivage);

        sc.close();
    }
}
```

Listing 1.8 – Implémentation de la classe Produit

Conclusion

À l'issue de ce troisième travail pratique, nous avons pu asseoir nos connaissances fondamentales en programmation orientée objet avec Java. La manipulation des classes telles que `Point`, `CompteBancaire` ou `Produit` nous a permis de comprendre l'importance de l'encapsulation, l'utilité des constructeurs pour l'initialisation des objets, ainsi que le découpage logique des fonctionnalités en méthodes distinctes. En outre, la création d'exercices interactifs tels que le `JeuDevinette` a mis en évidence l'interaction entre les entrées utilisateurs (avec la classe Scanner) et la logique interne des objets.

Ces concepts constituent désormais une base solide pour aborder la suite de notre apprentissage en Java, notamment pour les notions avancées d'héritage, de polymorphisme et surtout la structuration de projets logiciels complexes réutilisables.

Yazid TAHIRI ALAOUI
ENSAH – 2025 – 2026